MODUL 16 · PRODUKSI & CAPSTONE

Build Rilis APK/AAB & Panduan Capstone Project

Mempersiapkan aplikasi untuk dirilis: versi aplikasi, nama dan application ID, ikon dan splash screen, permission, konfigurasi Supabase dengan --dart-define-from-file, keystore dan signing Gradle Kotlin DSL, obfuscation, build APK split-per-abi dan App Bundle, pengujian build rilis, gambaran Play Console dan iOS, serta panduan lengkap Capstone Project.

Durasi Pertemuan 16 (150 Menit) + Belajar Mandiri

Prasyarat Modul 14: Proyek Catatan Kuliah Bagian 2 (CRUD & Cache Offline)
Modul 15: Debugging & Testing

Capaian Pembelajaran

- Mengatur versi aplikasi (version: x.y.z+build), nama aplikasi, dan application ID yang unik.
- Mengonfigurasi launcher icon (termasuk adaptive icon) dan native splash screen dengan generator.
- Mendeklarasikan permission yang dibutuhkan di AndroidManifest.xml dan Info.plist, termasuk INTERNET untuk build rilis.
- Memisahkan konfigurasi rahasia dari kode dengan --dart-define-from-file dan .gitignore.
- Membuat keystore, key.properties, dan konfigurasi signing di build.gradle.kts, lalu memverifikasi tanda tangan APK.
- Mengompilasi APK split-per-abi dan App Bundle dengan obfuscation, lalu mengujinya di perangkat sungguhan.
- Menjelaskan alur internal testing di Google Play Console dan gambaran rilis iOS.
- Menyelesaikan Capstone Project sesuai fitur wajib, rubrik, timeline, dan deliverables.

Daftar Isi Modul

1. Mode Build & Versi Aplikasi
2. Nama Aplikasi & Application ID
3. App Icon & Splash Screen
4. Permission di AndroidManifest & Info.plist
5. Konfigurasi Lingkungan dengan --dart-define-from-file
6. Keystore & Konfigurasi Signing
7. Build APK & AAB, Obfuscation, Uji di Perangkat
8. Google Play Console & Gambaran iOS

9. Panduan Capstone Project
10. Tugas Praktikum
11. Kuis Pemahaman & Kunci Jawaban

1. Mode Build & Versi Aplikasi

Selama ini aplikasi selalu dijalankan dalam mode *debug*. Pengguna sungguhan tidak pernah melihat mode itu: mereka memasang build **release** yang lebih kecil, lebih cepat, dan sudah ditandatangani. Modul ini membahas semua langkah dari kode yang "jalan di laptop" sampai APK/AAB yang siap dibagikan, lalu ditutup dengan panduan Capstone Project.

Tiga mode build Flutter

Mode	Perintah	Ciri	Dipakai untuk
Debug	`flutter run`	Hot reload, assert aktif, pita DEBUG, lambat, ukuran besar	Pengembangan sehari-hari
Profile	`flutter run --profile`	Performa mendekati rilis, DevTools tetap bisa dipakai	Mengukur performa (Modul 15)
Release	`flutter run --release`, `flutter build apk`	Kode dikompilasi AOT, assert dan `kDebugMode` mati, tanpa hot reload	Dibagikan ke pengguna

Peta jalan rilis

1. Atur nomor versi (bagian ini).
2. Atur nama aplikasi dan package id (bagian 2).
3. Pasang ikon dan splash screen (bagian 3).
4. Periksa permission di `AndroidManifest.xml` (bagian 4).
5. Keluarkan kunci Supabase dari kode dengan `--dart-define-from-file` (bagian 5).
6. Buat keystore dan konfigurasi signing (bagian 6).
7. Build APK/AAB dengan obfuscation lalu uji di HP sungguhan (bagian 7).
8. Unggah ke Play Console untuk internal testing (bagian 8).

Versi aplikasi: version: x.y.z+build

Nomor versi diatur di satu tempat saja, yaitu baris `version` di `pubspec.yaml`:

`pubspec.yaml` (potongan)

```yaml
name: catatan_kuliah
description: "Aplikasi catatan kuliah dengan Supabase dan cache offline."
publish_to: 'none'

# format: versionName+versionCode
version: 1.2.0+7
```

Bagian	Contoh	Di Android menjadi	Dilihat oleh	Aturan
Nama versi (sebelum `+`)	`1.2.0`	`versionName`	Pengguna (di Play Store dan pengaturan HP)	Bebas, biasanya mengikuti semantic versioning
Nomor build (setelah `+`)	`7`	`versionCode`	Sistem Android dan Play Store	Bilangan bulat, **wajib naik** setiap unggahan baru

Semantic versioning (`MAYOR.MINOR.PATCH`) memudahkan pengguna dan tim memahami besarnya perubahan:

- **PATCH** (`1.2.0 → 1.2.1`): perbaikan bug tanpa fitur baru.
- **MINOR** (`1.2.1 → 1.3.0`): fitur baru, misalnya fitur pencarian catatan.
- **MAYOR** (`1.3.0 → 2.0.0`): perubahan besar, misalnya struktur database berubah total.

Nilai `versionCode` dan `versionName` otomatis dibaca oleh Gradle dari `pubspec.yaml`. Di `android/app/build.gradle.kts` hasil `flutter create` sudah ada baris berikut, jadi **jangan** mengubah angka di sana:

`android/app/build.gradle.kts` **(potongan bawaan)**

```
defaultConfig {
    applicationId = "id.ac.unsoed.catatan_kuliah"
    minSdk = flutter.minSdkVersion
    targetSdk = flutter.targetSdkVersion
    versionCode = flutter.versionCode
    versionName = flutter.versionName
}
```

Versi juga bisa ditimpa sekali jalan dari terminal, misalnya oleh CI:

```
flutter build apk --build-name=1.2.1 --build-number=8
```

Menampilkan versi di dalam aplikasi

Halaman “Tentang” atau footer halaman Profil biasanya menampilkan versi aplikasi. Ini membantu saat pengguna melapor bug (“saya pakai versi berapa?”). Gunakan package `package_info_plus`:

```
flutter pub add package_info_plus
```

lib/utils/versi.dart

```dart
// lib/utils/versi.dart
import 'package:package_info_plus/package_info_plus.dart';

/// Contoh hasil: "1.2.0 (build 7)"
Future<String> versiAplikasi() async {
  final info = await PackageInfo.fromPlatform();
  return '${info.version} (build ${info.buildNumber})';
}
```

Panggil `versiAplikasi()` di `initState` halaman Profil lalu tampilkan hasilnya dengan `FutureBuilder` atau `setState`, seperti pola di Modul 09.

> ⚠️ **Kesalahan umum**
>
> - Mengunggah build baru ke Play Console dengan nomor build yang sama. Play Console menolak: *"Version code 7 has already been used"*. Naikkan angka setelah `+`.
> - Menurunkan nomor build. Android menolak memasang APK dengan `versionCode` lebih kecil di atas versi yang sudah terpasang.
> - Mengubah `versionCode = 7` langsung di `build.gradle.kts`. Hapus angka manual itu; pubspec adalah satu-satunya sumber versi.

2. Nama Aplikasi & Application ID

Setiap aplikasi Android punya dua identitas: **nama tampilan** yang dilihat pengguna di bawah ikon, dan **application ID** (package id) yang unik di seluruh Play Store. Nama tampilan boleh diganti kapan saja; application ID **tidak boleh berubah** setelah aplikasi dipublikasikan.

1. Nama aplikasi (label)

Secara bawaan, nama aplikasi sama dengan nama proyek (`catatan_kuliah`), lengkap dengan garis bawah. Ganti atribut `android:label`:

`android/app/src/main/AndroidManifest.xml` **(potongan)**

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:label="Catatan Kuliah"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        ...
    </application>
</manifest>
```

Untuk iOS, ubah nilai `CFBundleDisplayName`:

`ios/Runner/Info.plist` **(potongan)**

```
<key>CFBundleDisplayName</key>
<string>Catatan Kuliah</string>
```

Nama yang tampil di *task switcher* Android saat aplikasi berjalan diambil dari properti `title` pada `MaterialApp`, jadi samakan juga: `MaterialApp(title: 'Catatan Kuliah', ...)`.

2. Application ID

Application ID ditulis seperti nama domain terbalik. Untuk tugas kuliah, gunakan pola `id.ac.unsoed.<nim_atau_kelompok>.<nama_app>` agar tidak bentrok dengan teman sekelas.

- Hanya huruf kecil, angka, dan garis bawah; dipisah titik; minimal dua segmen.
- Setiap segmen harus diawali huruf: `id.ac.unsoed.h1d022001.catatan` boleh, `id.ac.unsoed.2022.catatan` tidak.
- Hindari `com.example`: Play Console menolak application ID yang diawali `com.example`.

Cara paling mudah adalah menentukannya **saat membuat proyek** dengan opsi `--org`:

```
flutter create --org id.ac.unsoed.h1d022001 catatan_kuliah
# hasil: applicationId = "id.ac.unsoed.h1d022001.catatan_kuliah"
```

Untuk proyek yang sudah terlanjur dibuat, ubah `applicationId` di Gradle:

`android/app/build.gradle.kts` (potongan)

```
android {
    // namespace = package kode Kotlin/Java (lokasi MainActivity)
    namespace = "com.example.catatan_kuliah"

    defaultConfig {
        // applicationId = identitas unik di HP dan Play Store
        applicationId = "id.ac.unsoed.h1d022001.catatan_kuliah"
        // ...
    }
}
```

	`applicationId`	`namespace`
Fungsi	Identitas aplikasi di HP dan Play Store	Nama package kode Kotlin/Java dan class `R`
Boleh berbeda satu sama lain?	Ya. Mengubah `applicationId` saja sudah cukup untuk mengganti identitas aplikasi.	
Jika diubah	Dianggap aplikasi yang berbeda (data lama tidak ikut)	Harus memindahkan `MainActivity.kt` ke folder dan `package` yang sesuai

Jika Anda juga ingin mengubah `namespace`, sesuaikan baris `package` di `MainActivity.kt` dan pindahkan file tersebut ke folder yang cocok:

`android/app/src/main/kotlin/id/ac/unsoed/h1d022001/catatan_kuliah/MainActivity.kt`

```
package id.ac.unsoed.h1d022001.catatan_kuliah

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

Setelah mengubah application ID, lakukan `flutter clean` lalu jalankan ulang. Untuk iOS, bundle identifier diatur dari Xcode (Runner → Signing & Capabilities).

⚠️ **Kesalahan umum**

- Mengubah `namespace` tanpa memindahkan `MainActivity.kt`. Build berhasil, tetapi aplikasi langsung tertutup dengan error `ClassNotFoundException: ...MainActivity`.
- Mengubah application ID setelah aplikasi diunggah ke Play Console. Play Store akan menganggapnya aplikasi baru; pengguna lama tidak mendapat update.
- Satu kelas memakai `com.example.catatan_kuliah` yang sama. Saat dosen memasang beberapa APK, APK berikutnya menimpa APK sebelumnya (atau gagal dipasang karena tanda tangan berbeda).

3. App Icon & Splash Screen

Aplikasi profesional tidak boleh menampilkan ikon bawaan Flutter atau layar putih kosong saat pertama dibuka. Dua package generator membuat semua ukuran ikon dan splash screen untuk Android dan iOS secara otomatis dari satu gambar.

Menyiapkan aset gambar

```
catatan_kuliah/
├── assets/
|   ├── icon/
|   |   ├── icon.png              # 1024x1024, latar penuh (untuk iOS dan Android lama)
|   |   └── icon_foreground.png   # 1024x1024, latar TRANSPARAN, logo di tengah
|   └── splash/
|       └── logo.png              # logo transparan untuk splash
└── pubspec.yaml
```

- Gunakan PNG persegi minimal 1024×1024 piksel. Jangan memberi sudut membulat sendiri; Android dan iOS akan memotongnya sesuai bentuk masing-masing.
- Untuk *adaptive icon* Android, logo pada `icon_foreground.png` harus berada di area tengah (sekitar 66% bagian dalam), karena pinggirannya bisa terpotong menjadi lingkaran, kotak membulat, atau bentuk lain tergantung merek HP.

Langkah 1: ikon aplikasi dengan flutter_launcher_icons

```
flutter pub add dev:flutter_launcher_icons
```

Konfigurasi bisa ditulis di `pubspec.yaml` atau di file terpisah. File terpisah membuat `pubspec.yaml` tetap rapi:

`flutter_launcher_icons.yaml` (di root proyek)

```yaml
flutter_launcher_icons:
  # Android: ganti ikon bawaan (ic_launcher) dengan ikon baru
  android: true
  ios: true
  image_path: "assets/icon/icon.png"           # 1024x1024, tanpa sudut membulat
  min_sdk_android: 21
  # Adaptive icon (Android 8+): background warna + foreground transparan
  adaptive_icon_background: "#1E3A8A"
  adaptive_icon_foreground: "assets/icon/icon_foreground.png"
  adaptive_icon_monochrome: "assets/icon/icon_foreground.png" # ikon bertema Android 13+
  # iOS menolak ikon transparan di App Store
  remove_alpha_ios: true
  background_color_ios: "#1E3A8A"
```

```
dart run flutter_launcher_icons
```

Output yang diharapkan:

```
• Creating colors.xml file and adding it to your Android project
• Overwriting the default Android launcher icon with a new icon
• Creating mipmap xml file Android
• Overwriting default iOS launcher icon with new icon

✓ Successfully generated launcher icons
```

Opsi	Fungsi
`android: true`	Timpa ikon bawaan `ic_launcher`. Isi dengan nama lain (misalnya `"launcher_icon"`) jika ingin membuat ikon baru tanpa menghapus yang lama.
`image_path`	Gambar sumber untuk semua platform
`adaptive_icon_background` / `_foreground`	Dua lapisan adaptive icon Android 8+. Keduanya wajib diisi agar adaptive icon dibuat.
`adaptive_icon_monochrome`	Versi satu warna untuk fitur *themed icons* Android 13+
`remove_alpha_ios`	Hapus transparansi ikon iOS (syarat App Store)

Langkah 2: splash screen dengan flutter_native_splash

Splash screen native tampil *sebelum* Flutter selesai dimuat, menggantikan layar putih kosong.

```
flutter pub add dev:flutter_native_splash
```

`flutter_native_splash.yaml` **(di root proyek)**

```yaml
flutter_native_splash:
  # Android 11 ke bawah dan iOS
  color: "#1E3A8A"
  image: assets/splash/logo.png
  color_dark: "#0F172A"
  image_dark: assets/splash/logo.png

  # Android 12+ punya aturan splash sendiri
  android_12:
    color: "#1E3A8A"
    image: assets/splash/logo.png
    color_dark: "#0F172A"
    image_dark: assets/splash/logo.png

  android: true
  ios: true
  web: false
```

```
dart run flutter_native_splash:create

# Mengembalikan ke splash bawaan jika ingin membatalkan
dart run flutter_native_splash:remove
```

Kenapa ada blok `android_12` terpisah? Mulai Android 12, sistem selalu menampilkan splash berupa **ikon di tengah lingkaran** di atas satu warna latar. Gambar penuh layar tidak didukung. Siapkan logo yang tetap terlihat baik jika dipotong menjadi lingkaran (sekitar 1152×1152 px dengan logo di dalam lingkaran berdiameter 768 px).

Langkah 3 (opsional): menahan splash selama inisialisasi

Aplikasi Catatan Kuliah perlu menginisialisasi Supabase dan Hive sebelum halaman pertama tampil. Splash bisa ditahan sampai proses itu selesai. Untuk fitur ini, package harus dipindah ke `dependencies` biasa (bukan `dev`), karena dipakai di kode aplikasi:

```
flutter pub remove flutter_native_splash
flutter pub add flutter_native_splash
```

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:flutter_native_splash/flutter_native_splash.dart';

Future<void> main() async {
  final binding = WidgetsFlutterBinding.ensureInitialized();
  // Tahan splash selama inisialisasi (Supabase, Hive, dll.)
  FlutterNativeSplash.preserve(widgetsBinding: binding);

  await Future.delayed(const Duration(seconds: 1)); // ganti dengan inisialisasi asli

  FlutterNativeSplash.remove(); // splash hilang, frame pertama tampil
  runApp(const MaterialApp(home: Scaffold(body: Center(child: Text('Siap!')))));
}
```

`preserve()` menahan splash native, `remove()` menghilangkannya. Jangan lupa memanggil `remove()` juga pada jalur error (misalnya di blok `catch`), agar aplikasi tidak “macet” di splash selamanya.

Melihat hasilnya

Ikon dan splash adalah bagian native, jadi **hot reload tidak cukup**. Hentikan aplikasi, hapus aplikasi lama dari emulator (beberapa launcher menyimpan cache ikon), lalu jalankan ulang `flutter run`.

⚠️ **Kesalahan umum**

- Logo foreground mengisi seluruh gambar 1024×1024, sehingga terpotong di HP dengan ikon berbentuk lingkaran.
- Menulis konfigurasi di dua tempat (di `pubspec.yaml` dan file terpisah). Pilih salah satu.
- Lupa menjalankan ulang perintah generator setelah mengganti gambar.
- Memakai `FlutterNativeSplash.preserve` sementara package masih di `dev_dependencies`. `flutter analyze` memberi peringatan `depend_on_referenced_packages`.

4. Permission di AndroidManifest & Info.plist

Android melindungi fitur sensitif (internet, kamera, lokasi, notifikasi) dengan **permission**. Aplikasi harus mendeklarasikan permission di `AndroidManifest.xml`; sebagian juga harus diminta langsung kepada pengguna saat aplikasi berjalan.

Jebakan nomor satu: permission INTERNET

Aplikasi Catatan Kuliah berjalan lancar di emulator, tetapi setelah APK release dipasang di HP, login Supabase selalu gagal. Kenapa? Proyek Flutter punya **tiga** file manifest:

```
android/app/src/
├── debug/AndroidManifest.xml     # dipakai saat flutter run (debug)
├── profile/AndroidManifest.xml   # dipakai saat --profile
└── main/AndroidManifest.xml      # dipakai SEMUA mode, termasuk release
```

Manifest `debug` bawaan sudah berisi permission INTERNET (dibutuhkan untuk hot reload), sedangkan manifest `main` **tidak**. Jadi aplikasi bisa mengakses internet saat debug, tetapi tidak saat release. Isi manifest debug bawaan:

`android/app/src/debug/AndroidManifest.xml` (bawaan flutter create)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- The INTERNET permission is required for development. Specifically,
         the Flutter tool needs it to communicate with the running application
         to allow setting breakpoints, to provide hot reload, etc.
    -->
    <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

Solusinya, tambahkan permission di manifest `main`, **di luar** tag `<application>`:

`android/app/src/main/AndroidManifest.xml` (potongan)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Wajib untuk Supabase, REST API, gambar dari internet -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="Catatan Kuliah"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        ...
    </application>
</manifest>
```

Permission lain yang sering dipakai

Fitur	Permission	Perlu diminta saat runtime?
Internet	`android.permission.INTERNET`	Tidak
Cek status koneksi	`android.permission.ACCESS_NETWORK_STATE`	Tidak
Kamera	`android.permission.CAMERA`	Ya
Lokasi	`android.permission.ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`	Ya
Notifikasi (Android 13+)	`android.permission.POST_NOTIFICATIONS`	Ya
Getar	`android.permission.VIBRATE`	Tidak

Contoh manifest untuk aplikasi capstone yang memakai kamera dan lokasi (misalnya aplikasi pengaduan sarana kampus):

`android/app/src/main/AndroidManifest.xml` (potongan)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

    <!-- Kamera tidak wajib: aplikasi tetap bisa dipasang di perangkat tanpa kamera -->
    <uses-feature android:name="android.hardware.camera" android:required="false"/>

    <application ...>
        ...
    </application>
</manifest>
```

Permission “berbahaya” (kamera, lokasi, notifikasi) juga harus diminta saat runtime. Package seperti `image_picker` dan `geolocator` biasanya sudah menampilkan dialog izin sendiri. Untuk kontrol penuh, gunakan package `permission_handler` dan ikuti petunjuk konfigurasinya di pub.dev.

iOS: alasan penggunaan di Info.plist

iOS tidak memakai daftar permission, tetapi mewajibkan **kalimat alasan** untuk setiap fitur sensitif. Tanpa kalimat ini aplikasi langsung tertutup saat meminta izin, dan App Store menolak aplikasi.

`ios/Runner/Info.plist` (potongan)

```
<key>NSCameraUsageDescription</key>
<string>Kamera dipakai untuk memotret fasilitas yang rusak.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Lokasi dipakai untuk menandai posisi laporan di peta kampus.</string>
```

> ⚠️ **Kesalahan umum**
>
> - Menambahkan INTERNET hanya di manifest `debug`, atau menaruh `<uses-permission>` di dalam tag `<application>`.
> - Meminta permission yang tidak dipakai. Play Console akan menanyakan alasannya, dan pengguna jadi curiga.
> - Tidak menangani kasus pengguna menolak izin. Tampilkan pesan dan jalan alternatif, jangan biarkan aplikasi crash.

5. Konfigurasi Lingkungan dengan --dart-define-from-file

Contoh singkat sering menulis URL dan key Supabase langsung di `main.dart`. Proyek Modul 13-14 sudah memakai cara yang dibahas di sini, jadi bagian ini merangkum dan memperdalamnya. Begitu kode di-push ke GitHub publik, kunci itu ikut terbaca semua orang. Bagian ini memindahkan konfigurasi ke file terpisah yang tidak masuk git, lalu menyuntikkannya saat build dengan `--dart-define-from-file`.

Konsep: konstanta saat kompilasi

Flag `--dart-define` mengirim pasangan kunci-nilai ke compiler. Kode Dart membacanya dengan `String.fromEnvironment`:

```
flutter run --dart-define=APP_ENV=dev --dart-define=SUPABASE_URL=https://abcdefghijkl.supabase.co
```

Menulis banyak `--dart-define` tentu merepotkan. Flutter menyediakan `--dart-define-from-file` yang membaca semua nilai dari satu file JSON (atau `.env`).

Langkah 1: file konfigurasi per lingkungan

`config/dev.json` (TIDAK di-commit)

```
{
  "SUPABASE_URL": "https://abcdefghijkl.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxx",
  "APP_ENV": "dev",
  "ENABLE_LOG": true
}
```

`config/prod.json` (TIDAK di-commit)

```
{
  "SUPABASE_URL": "https://abcdefghijkl.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxx",
  "APP_ENV": "prod",
  "ENABLE_LOG": false
}
```

Agar anggota kelompok tahu kunci apa saja yang dibutuhkan, commit file **contoh** tanpa nilai asli:

`config/example.json` (di-commit)

```
{
  "SUPABASE_URL": "https://ISI_PROJECT_ID.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "ISI_PUBLISHABLE_KEY_DARI_DASHBOARD",
  "APP_ENV": "dev",
  "ENABLE_LOG": true
}
```

Lalu tambahkan ke `.gitignore` di root proyek:

`.gitignore` (tambahkan di bagian bawah)

```
# Konfigurasi rahasia
/config/*.json
!/config/example.json
```

Baris `!` berarti pengecualian: semua file JSON di `config/` diabaikan kecuali `example.json`.

Langkah 2: class Env

`lib/config/env.dart`

```dart
// lib/config/env.dart
class Env {
  // Nilai diisi saat build lewat --dart-define atau --dart-define-from-file.
  // WAJIB const, karena nilainya ditanam ke aplikasi saat kompilasi.
  static const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');
  static const appEnv = String.fromEnvironment('APP_ENV', defaultValue: 'dev');
  static const enableLog = bool.fromEnvironment('ENABLE_LOG');

  static bool get isProduction => appEnv == 'prod';

  /// false jika aplikasi dijalankan tanpa --dart-define-from-file.
  static bool get isConfigured => supabaseUrl.isNotEmpty && supabaseKey.isNotEmpty;
}
```

- `String.fromEnvironment` dan `bool.fromEnvironment` **harus** dipanggil sebagai `const`. Dokumentasi Dart hanya menjamin nilainya terbaca dalam konteks `const`; tanpa `const`, nilainya bisa kosong, terutama di build release.
- Nilai JSON `true` / `false` dibaca dengan `bool.fromEnvironment`; angka dengan `int.fromEnvironment`.
- `defaultValue` dipakai jika kunci tidak ada di file.

Langkah 3: memakai Env di main.dart

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Lupa --dart-define-from-file? Tampilkan pesan yang jelas, jangan crash.
  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }
```

```dart
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabaseKey,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Catatan Kuliah',
      // Pita "DEBUG" hanya muncul jika bukan build produksi
      debugShowCheckedModeBanner: !Env.isProduction,
      home: Scaffold(
        appBar: AppBar(title: const Text('Catatan Kuliah')),
        body: Center(child: Text('Lingkungan: ${Env.appEnv}')),
      ),
    );
  }
}

class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                Icon(Icons.settings_suggest, size: 64, color: Colors.orange),
                SizedBox(height: 16),
                Text(
                  'Konfigurasi Supabase belum diisi',
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                ),
                SizedBox(height: 8),
                Text(
                  'Jalankan dengan:\n'
                  'flutter run --dart-define-from-file=config/dev.json',
                  textAlign: TextAlign.center,
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

Jika aplikasi dijalankan tanpa file konfigurasi (misalnya menekan tombol Run tanpa `launch.json`), pengguna melihat pesan jelas, bukan crash misterius dari Supabase:



Dijalankan tanpa `--dart-define-from-file` : ConfigErrorApp tampil

Langkah 4: menjalankan dan mem-build

```
# Pengembangan
flutter run --dart-define-from-file=config/dev.json

# Build rilis dengan konfigurasi produksi
flutter build apk --split-per-abi --dart-define-from-file=config/prod.json

# Juga bisa memakai file .env (format KUNCI=nilai)
flutter run --dart-define-from-file=.env
```

Agar tombol Run/Debug di VS Code otomatis memakai file konfigurasi, isi `.vscode/launch.json` seperti di Modul 15 bagian 2 (`"args": ["--dart-define-from-file=config/dev.json"]`).

ℹ️ Apakah key Supabase jadi rahasia?

Tidak sepenuhnya. Nilai `--dart-define` ditanam ke dalam APK, sehingga orang yang membongkar APK masih bisa menemukannya. Tujuan langkah ini adalah **tidak menyebarkan kunci lewat GitHub** dan memudahkan berganti project dev/prod. Keamanan data Supabase tetap bergantung pada **Row Level Security** (Modul 12). Publishable/anon key memang dirancang boleh berada di aplikasi klien; yang **tidak boleh sama sekali** masuk ke aplikasi adalah `service_role` /secret key.

⚠️ Kesalahan umum

- Menulis `final url = String.fromEnvironment('SUPABASE_URL');` tanpa `const`. Di build release, nilainya bisa kosong.
- Lupa menambahkan flag saat `flutter build`. APK jadi tidak bisa login. Simpan perintah build di README agar tidak lupa.
- Sudah terlanjur commit key ke GitHub, lalu hanya menghapus file di commit berikutnya. Key tetap ada di riwayat git. Buat ulang (rotate) key di dashboard Supabase.
- Memasukkan `service_role` key ke aplikasi agar “RLS tidak mengganggu”. Siapa pun yang membongkar APK bisa membaca dan menghapus seluruh database.

6. Keystore & Konfigurasi Signing

Android hanya mau memasang APK yang **ditandatangani secara digital**. Selama ini Flutter diam-diam memakai *debug key* yang sama untuk semua developer di dunia. Untuk rilis, Anda butuh kunci sendiri (**keystore**) yang menjadi bukti bahwa update aplikasi benar-benar berasal dari Anda.

Konsep: kenapa tanda tangan penting?

- Android hanya mengizinkan update jika APK baru ditandatangani dengan kunci yang **sama** dengan APK yang sudah terpasang.
- Jika keystore hilang, Anda tidak bisa lagi memperbarui aplikasi yang sudah dibagikan (kecuali memakai Play App Signing, lihat bagian 8).
- Jika keystore bocor, orang lain bisa membuat "update" palsu atas nama Anda.

Langkah 1: membuat keystore dengan keytool

`keytool` sudah tersedia bersama JDK yang dipasang Android Studio. Jalankan dari folder root proyek:

```
# macOS / Linux: simpan di android/app/
keytool -genkey -v -keystore android/app/upload-keystore.jks \
  -keyalg RSA -keysize 2048 -validity 10000 -alias upload

# Windows (PowerShell), satu baris:
keytool -genkey -v -keystore android\app\upload-keystore.jks -keyalg RSA -keysize 2048 -validity
10000 -alias upload
```

Jika muncul `keytool: command not found`, pakai keytool bawaan Android Studio. Lokasinya bisa dilihat dengan `flutter doctor -v` pada baris *Java binary at*, lalu ganti `java` dengan `keytool` pada path tersebut.

keytool akan menanyakan password dan identitas Anda:

```
Enter keystore password:  ********
Re-enter new password:    ********
What is your first and last name?
  [Unknown]:  Budi Santoso
What is the name of your organizational unit?
  [Unknown]:  Informatika
What is the name of your organization?
  [Unknown]:  UNSOED
What is the name of your City or Locality?
  [Unknown]:  Purwokerto
What is the name of your State or Province?
  [Unknown]:  Jawa Tengah
What is the two-letter country code for this unit?
  [Unknown]:  ID
Is CN=Budi Santoso, OU=Informatika, O=UNSOED, L=Purwokerto, ST=Jawa Tengah, C=ID correct?
  [no]:  yes
```

```
...
[Storing android/app/upload-keystore.jks]
```

Opsi	Arti
`-keystore`	Lokasi file keystore yang dibuat
`-alias upload`	Nama kunci di dalam keystore (satu keystore bisa berisi beberapa kunci)
`-validity 10000`	Berlaku 10.000 hari (sekitar 27 tahun). Play Store mensyaratkan masa berlaku panjang.
`-keyalg RSA -keysize 2048`	Algoritma dan panjang kunci

Langkah 2: file android/key.properties

Gradle membaca password dari file terpisah agar tidak tertulis di `build.gradle.kts`:

`android/key.properties`

```
storePassword=password_keystore_anda
keyPassword=password_key_anda
keyAlias=upload
# Relatif terhadap folder android/app/
storeFile=upload-keystore.jks
```

Periksa `android/.gitignore`. Proyek dari `flutter create` versi terbaru sudah mengabaikan file-file ini:

`android/.gitignore` **(bawaan, bagian bawah)**

```
# Remember to never publicly share your keystore.
# See https://flutter.dev/to/reference-keystore
key.properties
**/*.keystore
**/*.jks
```

Pastikan dengan perintah berikut. Jika file muncul di daftar “Untracked files”, berarti belum diabaikan:

```
git status --ignored android/ | grep -E "key.properties|jks"
# hasil yang benar: kedua file muncul di bagian "Ignored files"
```

Langkah 3: konfigurasi signing di build.gradle.kts

Proyek Flutter baru memakai Kotlin DSL (`build.gradle.kts`). Berikut file lengkap hasil `flutter create` versi terbaru yang sudah ditambah konfigurasi signing. Bagian yang Anda tambahkan:

dua baris `import` di paling atas, blok pembacaan `key.properties`, blok `signingConfigs`, dan isi `buildTypes`.

`android/app/build.gradle.kts`

```
import java.io.FileInputStream
import java.util.Properties

plugins {
    id("com.android.application")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
    id("dev.flutter.flutter-gradle-plugin")
}

// Baca android/key.properties (jika ada)
val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

android {
    namespace = "id.ac.unsoed.catatan_kuliah"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    defaultConfig {
        // TODO: Specify your own unique Application ID
(https://developer.android.com/studio/build/application-id.html).
        applicationId = "id.ac.unsoed.catatan_kuliah"
        // You can update the following values to match your application needs.
        // For more information, see: https://flutter.dev/to/review-gradle-config.
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        // Uses the version code from pubspec.yaml. When using split APKs, 1000 * ABI_VERSION
        // is added automatically by Flutter. (https://developer.android.com/studio/build/configure-
apk-splits#configure-APK-versions)
        // You can force using the value of versionCode by specifying the `-P force-version-code-
ignoring-abi=true`
        // flag during build.
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }

    signingConfigs {
        create("release") {
            if (keystorePropertiesFile.exists()) {
                keyAlias = keystoreProperties["keyAlias"] as String
                keyPassword = keystoreProperties["keyPassword"] as String
                storeFile = file(keystoreProperties["storeFile"] as String)
                storePassword = keystoreProperties["storePassword"] as String
            }
        }
```

```
    }

    buildTypes {
        release {
            signingConfig = if (keystorePropertiesFile.exists()) {
                signingConfigs.getByName("release")
            } else {
                // Belum ada key.properties: pakai debug key agar
                // `flutter run --release` tetap bisa dicoba
                signingConfigs.getByName("debug")
            }
        }
    }
}

kotlin {
    compilerOptions {
        jvmTarget = org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17
    }
}

flutter {
    source = "../.."
}
```

Penjelasan:

- `import` **harus di baris paling atas**, sebelum blok `plugins`. Di Kotlin script, import di tengah file menyebabkan error kompilasi Gradle.
- `rootProject.file("key.properties")` menunjuk ke `android/key.properties`, sedangkan `file(...)` di dalam blok `android` relatif terhadap `android/app/`. Itu sebabnya `storeFile=upload-keystore.jks` cukup ditulis nama filenya.
- Pemeriksaan `keystorePropertiesFile.exists()` membuat proyek tetap bisa di-build oleh anggota kelompok yang tidak memegang keystore (APK-nya memakai debug key). Tanpa pemeriksaan ini, `flutter run` biasa pun gagal di komputer mereka.
- Bagian lain (`compileSdk`, `versionCode`, blok `kotlin`) biarkan sesuai bawaan proyek Anda. Versi Flutter yang berbeda bisa menghasilkan isi bawaan yang sedikit berbeda.

Jika proyek Anda masih memakai `build.gradle` (Groovy, proyek lama), ikuti panduan resmi: docs.flutter.dev/deployment/android.

Langkah 4: memastikan APK benar-benar memakai kunci rilis

Setelah build (bagian 7), periksa sertifikat APK dengan `apksigner` dari Android SDK build-tools:

```
# macOS (sesuaikan versi build-tools di komputer Anda)
~/Library/Android/sdk/build-tools/<versi>/apksigner verify --print-certs \
  build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

```
# Benar: memakai keystore Anda
V2 Signer: certificate DN: CN=Budi Santoso, OU=Informatika, O=UNSOED, L=Purwokerto, ST=Jawa Tengah,
```

```
C=ID

# Salah: masih debug key (key.properties tidak terbaca)
V2 Signer: certificate DN: C=US, O=Android, CN=Android Debug
```

> 💡 **Backup keystore**
>
> Simpan salinan `upload-keystore.jks` beserta password-nya di tempat aman di luar laptop (password manager, Google Drive pribadi yang tidak dibagikan). Laptop rusak tanpa backup keystore berarti aplikasi Anda tidak bisa di-update lagi di luar Play App Signing.

> ⚠️ **Kesalahan umum**
>
> - Meng-commit `key.properties` atau file `.jks` ke GitHub publik.
> - Menulis path Windows dengan backslash tunggal di `key.properties` (`C:\Users\budi\key.jks`). Gunakan garis miring biasa (`C:/Users/budi/key.jks`) atau simpan keystore di `android/app/` dan tulis nama filenya saja.
> - Lupa mengganti `signingConfig` di `buildTypes.release`, sehingga APK rilis tetap memakai debug key.
> - Membuat keystore baru setiap kali build. Satu aplikasi = satu keystore selamanya.

7. Build APK & AAB, Obfuscation, Uji di Perangkat

Semua persiapan sudah selesai. Saatnya mengompilasi aplikasi menjadi file yang bisa dipasang: **APK** untuk dibagikan langsung, atau **AAB** (Android App Bundle) untuk diunggah ke Google Play.

APK atau AAB?

	APK	AAB
Bisa langsung dipasang di HP?	Ya	Tidak
Untuk Google Play	Tidak diterima untuk aplikasi baru	**Wajib**
Cara kerja	Satu file berisi kode untuk CPU tertentu (atau semua CPU)	Play Store membuat APK yang pas untuk setiap HP pengunduh
Cocok untuk	Dibagikan ke dosen/teman, pengumpulan tugas	Publikasi di Play Store

1. Membersihkan dan memeriksa sebelum build

```
# Pastikan kode bersih dan semua test lulus (Modul 15)
flutter analyze
flutter test

# Opsional: hapus hasil build lama jika ada perilaku aneh
flutter clean
flutter pub get
```

2. Build APK per arsitektur CPU

```
flutter build apk --split-per-abi --dart-define-from-file=config/prod.json
```

Contoh output asli untuk aplikasi yang memakai Supabase dan beberapa package lain:

```
Running Gradle task 'assembleRelease'...                          91.4s
✓ Built build/app/outputs/flutter-apk/app-armeabi-v7a-release.apk (13.2MB)
✓ Built build/app/outputs/flutter-apk/app-arm64-v8a-release.apk (16.1MB)
✓ Built build/app/outputs/flutter-apk/app-x86_64-release.apk (17.6MB)
```

Bandingkan dengan `flutter build apk` tanpa `--split-per-abi` pada proyek yang sama, yang menghasilkan satu *fat APK* berisi kode untuk semua CPU:

```
✓ Built build/app/outputs/flutter-apk/app-release.apk (47.2MB)
```

File	Untuk perangkat
`app-arm64-v8a-release.apk`	Hampir semua HP Android modern (pilih ini jika ragu)
`app-armeabi-v7a-release.apk`	HP Android lama/murah dengan CPU 32-bit
`app-x86_64-release.apk`	Emulator di laptop Intel/AMD, Chromebook
`app-release.apk` (fat APK)	Semua perangkat, tetapi ukurannya hampir tiga kali lipat

Flutter juga otomatis menambahkan `1000 × nomor ABI` ke `versionCode` setiap APK split, sehingga ketiga APK tidak bentrok nomor versinya.

3. Obfuscation dan split-debug-info

Kode Dart di APK bisa dibongkar untuk melihat nama class dan fungsi. `--obfuscate` mengacak nama-nama tersebut, dan `--split-debug-info` memisahkan simbol debug ke folder di laptop Anda sehingga APK lebih kecil. Keduanya selalu dipakai bersamaan.

```
flutter build apk --split-per-abi \
  --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
```

Folder `build/symbols/` berisi file simbol per arsitektur:

```
build/symbols/
├── app.android-arm.symbols
├── app.android-arm64.symbols
└── app.android-x64.symbols
```

Stack trace dari aplikasi yang di-obfuscate tidak bisa dibaca langsung. Simpan folder simbol untuk setiap versi yang dirilis, lalu terjemahkan stack trace dengan:

```
# stack_trace.txt = isi error yang disalin dari logcat/laporan crash
flutter symbolize -i stack_trace.txt -d build/symbols/app.android-arm64.symbols
```

> ⚠️ **Efek samping obfuscation**
>
> Nama class ikut diacak, sehingga kode yang bergantung pada nama tipe akan berubah perilaku, misalnya `runtimeType.toString()` dan `toString()` pada enum. Jangan menyimpan atau membandingkan data berdasarkan nilai-nilai tersebut; gunakan string tetap yang Anda tulis sendiri.

4. Build App Bundle untuk Google Play

```
flutter build appbundle \
  --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
```

```
Running Gradle task 'bundleRelease'...                             19.8s
✓ Built build/app/outputs/bundle/release/app-release.aab (45.1MB)
```

Ukuran AAB terlihat besar karena berisi semua arsitektur. Pengguna Play Store hanya mengunduh bagian yang sesuai dengan HP-nya, sehingga ukuran unduhannya mirip APK split.

5. Menguji build rilis di HP sungguhan

Build rilis **wajib** diuji sebelum dikumpulkan, karena beberapa bug hanya muncul di mode release (permission INTERNET, konfigurasi kosong, kode di dalam `assert`).

```
# Cara 1: jalankan langsung dalam mode release (HP tersambung USB, USB debugging aktif)
flutter run --release --dart-define-from-file=config/prod.json

# Cara 2: pasang APK yang sudah di-build ke perangkat yang tersambung
flutter install --use-application-binary=build/app/outputs/flutter-apk/app-arm64-v8a-release.apk

# Cara 3: lewat adb
adb install -r build/app/outputs/flutter-apk/app-arm64-v8a-release.apk

# Melihat log aplikasi yang sedang berjalan
flutter logs
```

Cara 4: kirim file APK ke HP (Google Drive, Telegram, kabel USB), buka dari aplikasi File, lalu izinkan “Instal dari sumber tidak dikenal” bila diminta.

Daftar uji minimal untuk build rilis Catatan Kuliah:

1. Aplikasi terbuka tanpa layar abu-abu; ikon, nama, dan splash sesuai.
2. Register, login, dan logout berhasil (membuktikan INTERNET dan konfigurasi Supabase benar).
3. Tambah, ubah, dan hapus catatan berhasil.

4. Matikan internet (mode pesawat): catatan dari cache Hive tetap tampil.
5. Tutup paksa lalu buka lagi: sesi login dan data cache masih ada.
6. Putar layar dan ubah ukuran font sistem ke terbesar: tidak ada overflow.

> ⚠️ **Kesalahan umum**
>
> - Mengirim `app-x86_64-release.apk` ke teman yang memakai HP biasa. Aplikasi tidak bisa dipasang (“App not installed”). Kirim `arm64-v8a`.
> - Memasang APK rilis di atas aplikasi debug yang sudah terpasang. Tanda tangannya berbeda sehingga gagal; hapus dulu aplikasi lama.
> - Mengumpulkan APK debug (`build/app/outputs/flutter-apk/app-debug.apk`). Ukurannya besar, lambat, dan menampilkan pita DEBUG.
> - Membuang folder `build/symbols` dengan `flutter clean` sebelum menyalinnya. Simpan di luar folder `build/` (misalnya `symbols/1.2.0+7/`, tetapi jangan di-commit).

8. Google Play Console & Gambaran iOS

Bagian ini memberi gambaran alur publikasi. Untuk mata kuliah ini, publikasi ke Play Store **tidak wajib**, tetapi memahami alurnya penting jika suatu hari Anda merilis aplikasi sungguhan atau bekerja di perusahaan.

Google Play Console

Akun developer Google Play dibuat di play.google.com/console dengan biaya pendaftaran satu kali dan verifikasi identitas. Setelah akun aktif, alurnya:

1. **Create app**: isi nama aplikasi, bahasa default, jenis (app/game), gratis atau berbayar.
2. **Lengkapi "Set up your app"**: kebijakan privasi (URL), akses aplikasi (jika butuh login, berikan akun demo untuk reviewer), iklan, rating konten, target usia, dan formulir **Data safety** (data apa yang dikumpulkan: email, catatan pengguna, dll.).
3. **Store listing**: deskripsi singkat dan lengkap, ikon 512×512, feature graphic 1024×500, minimal beberapa screenshot HP.
4. **Testing → Internal testing**: buat rilis baru, unggah file `app-release.aab`, isi catatan rilis.
5. **Tambahkan penguji**: daftar email (maksimal 100 orang). Penguji membuka tautan undangan, menerima, lalu memasang dari Play Store.
6. Setelah stabil, naikkan ke *closed testing*, lalu *production*. Akun developer pribadi baru biasanya diwajibkan menjalankan closed testing dengan sejumlah penguji selama periode tertentu sebelum boleh rilis production; cek syarat terbaru di Play Console.

Jalur	Penguji	Review Google	Kegunaan
Internal testing	Maks. 100 email	Sangat cepat / tanpa menunggu lama	Tim sendiri, dosen, asisten
Closed testing	Daftar email atau Google Group	Ya	Beta tester terbatas
Open testing	Siapa saja yang bergabung	Ya	Beta publik
Production	Semua pengguna Play Store	Ya	Rilis resmi

Play App Signing

Saat mengunggah AAB pertama, Play Console menawarkan **Play App Signing**: Google menyimpan kunci penandatanganan aplikasi yang sesungguhnya, sedangkan keystore Anda

menjadi *upload key* (itulah sebabnya aliasnya `upload`). Keuntungannya, jika upload key hilang, Anda bisa meminta reset ke Google tanpa kehilangan aplikasi.

Jika memakai fitur Google Sign-In atau API yang meminta sidik jari SHA-1/SHA-256, ambil nilainya dari halaman *App integrity* di Play Console (kunci aplikasi), bukan hanya dari keystore lokal. Sidik jari keystore lokal bisa dilihat dengan:

```
keytool -list -v -keystore android/app/upload-keystore.jks -alias upload
```

Setiap update

```
# 1. Naikkan versi di pubspec.yaml, misalnya 1.2.0+7 menjadi 1.2.1+8
# 2. Build ulang
flutter build appbundle --obfuscate --split-debug-info=build/symbols \
  --dart-define-from-file=config/prod.json
# 3. Unggah app-release.aab ke jalur testing, isi catatan rilis
# 4. Simpan folder build/symbols untuk versi ini
```

iOS secara singkat

Build iOS hanya bisa dilakukan di **macOS dengan Xcode**. Gambaran alurnya:

1. Daftar **Apple Developer Program** (berbayar tahunan). Tanpa akun berbayar, aplikasi hanya bisa dipasang ke iPhone sendiri lewat kabel dan berlaku beberapa hari.
2. Buka `ios/Runner.xcworkspace` di Xcode, atur *Bundle Identifier* dan *Team* di tab *Signing & Capabilities*.
3. Isi alasan permission di `Info.plist` (bagian 4).
4. Build arsip: `flutter build ipa --dart-define-from-file=config/prod.json`.
5. Unggah ke **App Store Connect** (lewat aplikasi Transporter atau Xcode), lalu bagikan ke penguji lewat **TestFlight**.
6. Kirim untuk review App Store saat siap rilis.

Android	Padanan iOS
Play Console	App Store Connect
Internal testing	TestFlight
applicationId	Bundle Identifier
Keystore + key.properties	Certificate + Provisioning Profile (dikelola Xcode)
`flutter build appbundle`	`flutter build ipa`
AndroidManifest permission	Info.plist usage description

⚠️ **Kesalahan umum**

- Mengunggah APK ke Play Console. Aplikasi baru wajib memakai AAB.
- Aplikasi butuh login tetapi tidak memberi akun demo ke reviewer, sehingga rilis ditolak.
- Formulir Data safety tidak sesuai kenyataan (misalnya menyatakan tidak mengumpulkan email padahal ada fitur register).
- Tidak punya kebijakan privasi. Untuk aplikasi yang menyimpan data pengguna, URL kebijakan privasi wajib ada (boleh berupa halaman GitHub Pages sederhana).

9. Panduan Capstone Project

> 🏆 **Ketentuan Capstone Project (Bobot 35% Nilai Akhir)**
>
> Mahasiswa secara berkelompok (2-3 orang) atau mandiri membangun satu aplikasi mobile lengkap yang menyelesaikan permasalahan nyata di lingkungan kampus atau masyarakat. Capstone adalah tempat menggabungkan semua materi: widget dan layout (Modul 03-06), navigasi (07), Provider (08), API dan penyimpanan (09-12), arsitektur proyek (13-14), testing (15), dan rilis (16).

Boleh memakai arsitektur Catatan Kuliah sebagai dasar

Anda **diizinkan dan disarankan** memakai struktur proyek Modul 13-14 sebagai kerangka: `AuthProvider` + `AuthGate` untuk login Supabase, pola `Repository` dengan remote data source (Supabase) dan cache Hive, serta Provider untuk state halaman. Syaratnya:

- Domain aplikasi **harus berbeda** dari catatan kuliah (tabel, model, dan halaman baru, bukan sekadar mengganti judul).
- Minimal **dua entitas data yang saling berhubungan** (misalnya `laporan` dan `kategori`, atau `produk` dan `pesanan`), bukan hanya satu tabel.
- Sebutkan di README bagian mana yang diadaptasi dari modul.

Struktur folder yang direkomendasikan:

```
lib/
├── main.dart              # inisialisasi Env, Supabase, Hive, MultiProvider
├── config/env.dart        # konfigurasi --dart-define-from-file
├── models/                # class data + fromJson/toJson
├── data/                  # data source (Supabase, Hive) dan repository
├── providers/             # AuthProvider dan provider fitur
├── pages/                 # satu file per halaman
├── widgets/               # widget yang dipakai ulang
└── utils/                 # validator, formatter, konstanta
test/
├── unit/                  # validator, model, provider
├── mock/                  # fake data source
└── widget/                # halaman login, daftar, form
```

Pilihan tema proyek

1. **Sistem Pengaduan Sarana Kampus:** laporan fasilitas rusak dengan foto kamera, kategori, dan status penanganan (baru, diproses, selesai).
2. **Marketplace UMKM Mahasiswa:** katalog produk, keranjang belanja (Provider), riwayat pesanan per pengguna.

3. **Perpustakaan & Peminjaman Buku:** pencarian buku via REST API publik, daftar pinjaman di Supabase, bookmark offline.
4. **Layanan Akademik:** jadwal kuliah, pengajuan surat izin online, pengingat tugas.
5. **Tema bebas** dengan persetujuan dosen paling lambat minggu ke-2 periode capstone.

Fitur wajib

No	Fitur	Keterangan / modul acuan
1	Autentikasi	Register, login, logout, sesi tetap tersimpan saat aplikasi dibuka ulang (Modul 13)
2	CRUD data online	Minimal 2 tabel berelasi di Supabase dengan Row Level Security aktif (Modul 12, 14)
3	Penyimpanan lokal	Cache offline (Hive/SQLite) atau preferensi pengguna (SharedPreferences) yang benar-benar dipakai (Modul 10, 11, 14)
4	State management	Provider dengan pemisahan UI, provider, dan repository (Modul 08, 14)
5	Navigasi	Minimal 5 halaman, ada passing data antarhalaman, ada bottom navigation atau drawer (Modul 07)
6	Form tervalidasi	Minimal 2 form dengan validasi dan pesan error yang jelas (Modul 05)
7	State UI lengkap	Setiap daftar menangani loading, kosong, error, dan offline
8	Testing	Minimal 10 test: unit test (validator/provider), test repository dengan fake, dan 2 widget test (Modul 15)
9	Siap rilis	Ikon dan splash sendiri, nama dan application ID sendiri, key Supabase lewat `--dart-define-from-file`, APK release bertanda tangan (Modul 16)

Nilai tambah (bonus maksimal 10 poin): fitur kamera/galeri dengan Supabase Storage, pencarian dan filter, mode gelap, realtime Supabase, notifikasi lokal, integration test, atau dipublikasikan di jalur internal testing Play Console.

Rubrik penilaian

Kriteria	Bobot	Sangat baik (85-100)	Cukup (60-75)	Kurang (< 60)
Fungsionalitas fitur wajib	25%	Semua fitur wajib berjalan di APK release tanpa crash	Sebagian kecil fitur belum lengkap atau ada bug minor	Beberapa fitur wajib tidak ada atau crash
Arsitektur & kualitas kode	20%	Lapisan UI/provider/repository jelas, `flutter analyze` 0 issue, nama dan struktur rapi	Ada pemisahan tetapi masih ada logika data di widget, beberapa lint	Semua logika di dalam widget, banyak warning
Integrasi data (online & offline)	15%	CRUD berelasi lancar, RLS benar, cache offline berfungsi	CRUD berjalan, offline terbatas	Data hanya lokal atau RLS dimatikan
UI/UX	15%	Material 3 konsisten, bebas overflow, state loading/kosong/error rapi, responsif	Tampilan cukup, ada overflow kecil atau state yang terlewat	Tampilan tidak konsisten, sering overflow
Testing	10%	≥ 10 test bermakna lulus, termasuk fake repository dan widget test	Beberapa test, sebagian hanya kasus sukses	Tidak ada test atau test gagal
Rilis & dokumentasi	10%	APK bertanda tangan, README lengkap sesuai template, tidak ada rahasia di repo	README kurang lengkap, APK debug	Tanpa README atau key Supabase ter-commit
Presentasi & demo	5%	Demo lancar, semua anggota menguasai kode saat tanya jawab	Demo tersendat, sebagian anggota kurang menguasai	Tidak bisa mendemonstrasikan aplikasi

Nilai individu dapat disesuaikan berdasarkan kontribusi yang terlihat di riwayat commit dan jawaban saat tanya jawab.

Timeline (6 minggu)

Minggu	Kegiatan	Luaran (dicek dosen/asisten)
1	Pembentukan kelompok, pemilihan tema, analisis kebutuhan	Proposal 1-2 halaman: masalah, fitur, sketsa layar
2	Desain data dan UI	Skema tabel Supabase + RLS, wireframe semua halaman, repo GitHub dibuat
3	Fondasi: auth, navigasi, model, repository	Login/register berjalan, kerangka halaman
4	Fitur inti: CRUD berelasi, cache offline	Demo progres 50% kepada asisten
5	Penyempurnaan UI, testing, perbaikan bug	Minimal 10 test lulus, `flutter analyze` bersih
6	Rilis dan dokumentasi	APK release, README, video demo, presentasi akhir

Deliverables (yang dikumpulkan)

1. **Repositori GitHub** (publik atau undang dosen): riwayat commit dari semua anggota, tanpa `key.properties`, `*.jks`, dan `config/*.json` berisi key asli.
2. **README.md** sesuai template di bawah.
3. **APK release** `app-arm64-v8a-release.apk` yang sudah ditandatangani, diunggah di bagian *Releases* GitHub.
4. **Video demo** 3-5 menit (YouTube unlisted/Google Drive): alur utama dari register sampai fitur inti, termasuk kondisi offline.
5. **Slide presentasi** (maksimal 10 slide): masalah, solusi, arsitektur, demo, tantangan, pembagian tugas.
6. **Akun demo** (email dan password) untuk penguji, ditulis di README atau formulir pengumpulan.

Template README.md

`README.md`

```
# Lapor Kampus

Aplikasi pelaporan kerusakan fasilitas kampus UNSOED. Mahasiswa memotret
fasilitas yang rusak, memilih kategori, dan memantau status penanganannya.

## Anggota Kelompok
| Nama          | NIM         | Peran                        |
```

```
|---------------|-------------|------------------------------|
| Budi Santoso  | H1D022001   | Auth, repository, testing    |
| Siti Aminah   | H1D022002   | UI halaman laporan, navigasi |
| Rizky Pratama | H1D022003   | Supabase (tabel, RLS), rilis |

## Screenshot
| Login | Daftar Laporan | Detail | Mode Offline |
|-------|----------------|--------|--------------|
| ![](docs/login.png) | ![](docs/list.png) | ![](docs/detail.png) | ![](docs/offline.png) |

## Fitur
- Register, login, logout (Supabase Auth), sesi tersimpan
- CRUD laporan + kategori (tabel berelasi, RLS aktif)
- Cache offline dengan Hive
- Unggah foto ke Supabase Storage (bonus)

## Arsitektur
UI (pages, widgets) -> Provider -> Repository -> Data source (Supabase / Hive).
Diadaptasi dari proyek Catatan Kuliah Modul 13-14: AuthGate, AuthProvider,
pola NoteRepository (diganti menjadi LaporanRepository).

## Skema Database
| Tabel    | Kolom penting                                |
|----------|----------------------------------------------|
| kategori | id, nama                                     |
| laporan  | id, user_id, kategori_id, judul, status, foto |

## Menjalankan Proyek
1. `flutter pub get`
2. Salin `config/example.json` menjadi `config/dev.json`, isi URL dan key Supabase
3. `flutter run --dart-define-from-file=config/dev.json`

## Testing
```bash
flutter test            # 14 test, semua lulus
flutter test --coverage
```

Build Rilis
```bash
flutter build apk --split-per-abi --obfuscate \
  --split-debug-info=build/symbols --dart-define-from-file=config/prod.json
```

Unduh APK
Lihat halaman Releases: app-arm64-v8a-release.apk (v1.0.0+1)

Video Demo
https://youtu.be/xxxxxxxx

Akun Demo
Email: demo@laporkampus.id / Password: diberikan di formulir pengumpulan
```

Panduan presentasi dan demo

- Demo memakai **APK release di HP sungguhan**, bukan emulator dalam mode debug.
- Tunjukkan satu skenario offline: matikan internet, data cache tetap tampil.

- Siapkan diri untuk pertanyaan seperti “tunjukkan di mana data laporan diambil dari Supabase” atau “apa yang terjadi jika token kedaluwarsa”. Setiap anggota harus bisa menjelaskan kode yang ia tulis.
- Boleh memakai bantuan AI atau referensi daring, tetapi sebutkan di README dan pastikan Anda memahami setiap baris kode yang dikumpulkan.

> ⚠️ **Kesalahan umum capstone**
>
> - Satu anggota mengerjakan semuanya dan anggota lain hanya membuat slide. Riwayat commit akan terlihat.
> - Mematikan RLS di Supabase agar “tidak error”. Ini mengurangi nilai integrasi data secara signifikan.
> - Hanya menguji di emulator debug, lalu APK release gagal login saat presentasi (lupa permission INTERNET atau lupa `--dart-define-from-file`).
> - Meng-commit `config/dev.json` berisi key asli. Rotasi key di dashboard Supabase jika sudah terlanjur.
> - Mulai mengerjakan di minggu ke-5. Ikuti timeline; demo progres minggu ke-4 adalah titik pemeriksaan wajib.

Tugas Praktikum: Praktikum 16: Rilis APK Catatan Kuliah & Persiapan Repositori Capstone

Sasaran Praktikum

Mengubah proyek Catatan Kuliah (Modul 13-14) menjadi APK release yang bertanda tangan, aman dari kebocoran key, dan siap dipasang di HP orang lain, sekaligus menyiapkan repositori capstone sesuai standar.

Langkah Pengerjaan

1. Ubah `version` di `pubspec.yaml` menjadi `1.0.0+1`, lalu tampilkan versi aplikasi di halaman Profil memakai `package_info_plus`.
2. Ganti `android:label` menjadi "Catatan Kuliah" dan `applicationId` menjadi `id.ac.unsoed.<nim>.catatan_kuliah`.
3. Buat ikon 1024×1024 (dengan versi foreground transparan) dan logo splash, lalu jalankan `dart run flutter_launcher_icons` dan `dart run flutter_native_splash:create`.
4. Tambahkan permission INTERNET di `android/app/src/main/AndroidManifest.xml`.
5. Pindahkan URL dan publishable key Supabase ke `config/dev.json` dan `config/prod.json`, buat `config/example.json`, perbarui `.gitignore`, dan gunakan class `Env`. Pastikan `git status` tidak menampilkan file konfigurasi rahasia.
6. Buat keystore dengan `keytool`, isi `android/key.properties`, dan tambahkan konfigurasi signing ke `android/app/build.gradle.kts`.
7. Build dengan `flutter build apk --split-per-abi --obfuscate --split-debug-info=build/symbols --dart-define-from-file=config/prod.json`, lalu verifikasi tanda tangan dengan `apksigner verify --print-certs` (screenshot hasilnya).
8. Pasang `app-arm64-v8a-release.apk` di HP fisik (milik sendiri atau teman) dan jalankan 6 butir daftar uji build rilis dari bagian 7. Catat hasilnya dalam tabel di README.
9. Unggah APK ke bagian Releases GitHub dan lengkapi README sesuai template capstone (screenshot, fitur, cara menjalankan, cara build).
10. Bonus: build `flutter build appbundle` dan bandingkan ukurannya dengan APK split dan fat APK.

Output yang Diharapkan

APK release bertanda tangan keystore sendiri (bukan Android Debug) yang terpasang di HP fisik dengan ikon, nama, dan splash kustom; login dan CRUD Supabase berjalan; repositori GitHub tanpa key/keystore, dengan README lengkap dan APK di halaman Releases.

Rubrik Penilaian

APK release berhasil dibuat dan bertanda tangan kunci rilis (30%), branding: ikon, splash, nama, application ID (20%), konfigurasi aman: dart-define-from-file dan .gitignore benar (20%), hasil uji di HP fisik terdokumentasi (15%), README dan Releases GitHub (15%).

Kuis Pemahaman

1. Pada baris version: 1.4.2+15 di pubspec.yaml, angka 15 berfungsi sebagai...

A. versionName yang dilihat pengguna di Play Store

B. versionCode, bilangan bulat yang wajib naik setiap kali mengunggah build baru

C. minSdk Android

D. Jumlah halaman aplikasi

2. Aplikasi Catatan Kuliah bisa login saat flutter run, tetapi APK release yang dipasang di HP selalu gagal terhubung ke Supabase. Penyebab yang paling mungkin adalah...

A. Supabase tidak mendukung Android

B. Permission INTERNET hanya ada di manifest debug, belum ditambahkan ke android/app/src/main/AndroidManifest.xml

C. APK release tidak bisa memakai package http

D. Ikon aplikasi belum diganti

3. Mengapa kode berikut bisa menghasilkan nilai kosong di build release? final url = String.fromEnvironment('SUPABASE_URL');

A. Karena nama kunci harus huruf kecil

B. Karena String.fromEnvironment hanya dijamin bekerja jika dipanggil dalam konteks const

C. Karena --dart-define hanya berlaku di iOS

D. Karena nilai harus dibaca dengan SharedPreferences

4. Apa tujuan utama memakai --dart-define-from-file=config/prod.json dengan config/*.json di .gitignore?

A. Membuat key Supabase mustahil ditemukan oleh siapa pun

B. Agar key tidak tersebar lewat repositori git dan konfigurasi dev/prod mudah ditukar tanpa mengubah kode

C. Agar APK lebih kecil

D. Agar RLS Supabase tidak diperlukan lagi

5. Mengapa file key.properties dan keystore (.jks) harus masuk .gitignore?

A. Karena GitHub tidak bisa menyimpan file biner

B. Karena siapa pun yang memegang keystore beserta password-nya bisa menandatangani update palsu atas nama Anda

C. Agar ukuran repositori di bawah 1 GB

D. Karena keystore hanya bisa dibaca Windows

6. Apa keuntungan flutter build apk --split-per-abi dibandingkan flutter build apk biasa?

A. Menghasilkan APK terpisah per arsitektur CPU sehingga setiap file jauh lebih kecil daripada fat APK

B. Membuat aplikasi bisa berjalan di iOS

C. Menghapus kebutuhan keystore

D. Mengubah APK menjadi AAB

7. Anda mem-build dengan --obfuscate --split-debug-info=build/symbols. Apa yang harus dilakukan agar stack trace crash dari pengguna bisa dibaca?

A. Tidak perlu apa-apa, stack trace selalu terbaca

B. Menyimpan folder simbol untuk versi tersebut dan menerjemahkan stack trace dengan flutter symbolize

C. Build ulang tanpa obfuscate lalu meminta pengguna mengulang crash

D. Menghapus folder build/symbols agar APK lebih aman

8. Format apa yang wajib diunggah ke Google Play Console untuk aplikasi baru, dan jalur apa yang paling cocok untuk membagikan ke dosen dan anggota kelompok?

A. APK, jalur production

B. AAB (Android App Bundle), jalur internal testing

C. IPA, jalur TestFlight

D. ZIP berisi kode sumber, jalur open testing

Kunci Jawaban & Pembahasan

1. B — Bagian sebelum + adalah versionName (1.4.2), bagian setelahnya adalah versionCode (15). Play Console menolak unggahan dengan versionCode yang sudah pernah dipakai, dan Android menolak downgrade.

2. B — flutter create menaruh permission INTERNET di manifest debug (untuk hot reload). Build release hanya memakai manifest main, sehingga permission harus ditambahkan di sana.

3. B — Nilai --dart-define ditanam saat kompilasi. Dokumentasi Dart hanya menjamin String.fromEnvironment terbaca dalam konteks const, sehingga tulis static const url = String.fromEnvironment('SUPABASE_URL');.

4. B — Nilai dart-define tetap tertanam di APK, jadi bukan rahasia mutlak. Manfaatnya adalah key tidak ikut ter-commit dan lingkungan mudah diganti. Keamanan data tetap bergantung pada RLS, dan service_role key tidak boleh ada di aplikasi.

5. B — Keystore adalah identitas kriptografis pemilik aplikasi. Android hanya menerima update yang ditandatangani dengan kunci yang sama, sehingga kebocoran keystore memungkinkan pemalsuan update.

6. A — APK biasa berisi kode native untuk semua CPU (pada contoh modul 47,2 MB). Split per ABI menghasilkan file terpisah seperti arm64-v8a (16,1 MB) yang cukup untuk hampir semua HP modern.

7. B — Obfuscation mengacak nama fungsi. File .symbols menyimpan pemetaan aslinya, sehingga flutter symbolize -i stack.txt -d build/symbols/app.android-arm64.symbols bisa mengembalikan nama yang terbaca.

8. B — Aplikasi baru di Play Store wajib memakai AAB. Internal testing memungkinkan hingga 100 penguji berdasarkan email dan prosesnya paling cepat, cocok untuk tim dan dosen.